

Creación y administración de un demonio con systemd

Práctica: **Demonio**

Materia: Sistemas Operativos

Alumno: Hector Manuel Carbajal Zamudio

Docente: Miguel Angel Barron Hernandez

Fecha: 29 de septiembre de 2026

1. Introducción

Gran parte del trabajo de un sistema operativo ocurre sin que el usuario lo vea: la red, las impresoras, los registros del sistema o las tareas programadas funcionan gracias a procesos que corren en segundo plano, llamados **demonios**. En Ubuntu, el encargado de iniciarlos, vigilarlos y detenerlos es `systemd`, el primer proceso del sistema (PID 1).

En esta práctica se escribió un script en Bash que se ejecuta indefinidamente y se registró como servicio de `systemd` (`mi-demonio.service`). Después se comprobó su funcionamiento, su consumo de recursos, su reinicio automático ante una terminación forzada y su desinstalación.

2. Objetivos

- Comprender el concepto de demonio y el papel de `systemd` como administrador de servicios.
- Crear una unidad de servicio y controlarla con `systemctl` y `journalctl`.
- Observar y justificar teóricamente el ciclo de vida del proceso, su consumo de recursos y su tolerancia a fallos.

3. Marco teórico

3.1 Demonios

Un demonio es un proceso que no está asociado a una terminal de control, se ejecuta en segundo plano y ofrece un servicio de forma continua. Tradicionalmente el propio programa se “demonizaba”: hacía `fork()`, el padre terminaba, el hijo llamaba a `setsid()` y cerraba sus descriptores. Con `systemd` esto ya no es necesario. El programa puede ser un simple bucle y `systemd` se encarga de desvincularlo de cualquier terminal, asignarle un usuario y capturar su salida en el *journal*.

3.2 `systemd` y las unidades

`systemd` describe cada servicio con un archivo de **unidad** con tres secciones principales:

- [Unit]: metadatos y dependencias.
- [Service]: qué ejecutar (ExecStart), con qué usuario (User) y qué hacer si termina (Restart).
- [Install]: en qué *target* se activa. multi-user.target equivale al sistema arrancado en modo multiusuario.

enable crea un enlace simbólico en multi-user.target.wants/, y --now además inicia el servicio de inmediato.

3.3 Control groups (cgroups)

systemd coloca cada servicio en su propio *cgroup*, un mecanismo del kernel que agrupa procesos para contabilizar y limitar sus recursos. Así systemd sabe exactamente qué procesos pertenecen al servicio, incluidos los hijos como sleep, y puede reportar su memoria y CPU o detenerlos a todos juntos.

3.4 Estados de un proceso y señales

Mientras ejecuta sleep 5, el proceso está en estado **bloqueado** (*S, interruptible sleep*): el planificador no le asigna CPU hasta que vence el temporizador. pkill envía la señal SIGTERM, que por defecto termina el proceso. Su padre, systemd, recibe la notificación de terminación (SIGCHLD) y aplica la política Restart.

4. Desarrollo

4.1 Script del demonio

```
#!/bin/bash
while true; do
    echo "Demonio funcionando: $(date)" >> /home/hector/mensaje_demonio.txt
    sleep 5
done
```

4.2 Unidad de servicio

```
[Unit]
Description=Mí primer demonio

[Service]
```

```
User=hector
ExecStart=/home/hector/mi_demonio.sh
Restart=always
```

```
[Install]
WantedBy=multi-user.target
```

4.3 Instalación, prueba y limpieza

```
$ chmod +x ~/mi_demonio.sh
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now mi-demonio
$ systemctl status mi-demonio
$ tail -f ~/mensaje_demonio.txt
$ sudo pkill -f mi_demonio.sh
$ systemctl status mi-demonio
$ journalctl -u mi-demonio -f
$ sudo systemctl stop mi-demonio && sudo systemctl disable mi-demonio
$ sudo rm /etc/systemd/system/mi-demonio.service && sudo systemctl daemon-reload
```

Los scripts de la carpeta Código/ (instalar_demonio.sh, probar_reinicio.sh y desinstalar_demonio.sh) automatizan estos pasos.

5. Observaciones

Datos tomados de las capturas en Terminal/capturas/, en el equipo HP-Papas:

Momento	Observación
09:26	Script creado con permisos -rwxrwxr-x (121 bytes).
09:28:21	enable --now crea el symlink y el servicio arranca: active (running), PID 17140, Tasks: 2, Memory: 2.6M (pico 2.9M), CPU: 62ms. En el <i>cgroup</i> aparecen /bin/bash /home/hector/mi_demonio.sh (17140) y sleep 5 (17485).
09:28:21 - 09:29:16	tail -f muestra 12 líneas, exactamente una cada 5 segundos.
09:29:32	Tras sudo pkill -f mi_demonio.sh, el <i>journal</i> registra Deactivated successfully, luego Scheduled restart job, restart counter is at 1 y Started. El nuevo proceso tiene PID 17555, Memory: 2M (pico 3.4M), CPU: 74ms.

Limpieza	Aparece el aviso <code>The unit file ... changed on disk. Run 'systemctl daemon-reload'</code> y se elimina el symlink de <code>multi-user.target.wants</code> .
----------	--

6. Análisis reflexivo del comportamiento del sistema operativo

6.1 Supervisión y tolerancia a fallos

El cambio de PID de 17140 a 17555 demuestra que el servicio no “sobrevivió” al `pkill`: el proceso original terminó y `systemd` creó uno **nuevo**. Esto es posible porque `systemd` es el padre del proceso y recibe la notificación de su terminación. Como la unidad declara `Restart=always`, programa un reinicio (el tiempo de espera por defecto es de 100 ms) y aumenta el contador de reinicios. Para el usuario, el servicio apenas estuvo caído una fracción de segundo. En el archivo de mensajes se nota solo un pequeño hueco. Este mismo mecanismo mantiene en pie servidores web o bases de datos en producción.

El *journal* dice `Deactivated successfully` y no *failed*. `SIGTERM` es una terminación “limpia” para `systemd`. Aun así, con `Restart=always` el servicio se reinicia sin importar el motivo. Con `Restart=on-failure` no se habría reiniciado en este caso. Elegir la política correcta depende de si una terminación limpia debe considerarse normal o no.

6.2 Consumo de recursos y planificación

Aunque el demonio corre “para siempre”, solo usó 62-74 ms de CPU. La razón es que pasa casi todo el tiempo en `sleep 5`, bloqueado a la espera de un temporizador del kernel. El planificador solo le asigna CPU unos microsegundos cada 5 s para ejecutar `echo` y `date`. En cambio, un bucle sin `sleep` ocuparía un núcleo completo al 100 %. Esto muestra la diferencia entre espera activa y espera bloqueada.

La memoria de 2-3 MB corresponde casi toda al intérprete `bash`. `Tasks: 2` refleja el modelo de procesos de Unix: cada comando externo (`sleep`) es un proceso hijo creado con `fork()` + `exec()`. Gracias al *cgroup*, `systemd` contabiliza ambos procesos como parte del servicio.

6.3 Separación entre configuración en disco y estado en memoria

El aviso `unit file ... changed on disk` muestra que `systemd` no lee el archivo de unidad en cada operación. Mantiene en memoria la versión que cargó con el último `daemon-reload`. Si el archivo se modifica o se borra antes de detener y deshabilitar el servicio, lo que hay en disco ya no coincide con lo que `systemd` administra. Por eso el orden correcto de desinstalación es `stop` → `disable` → `rm` → `daemon-reload`, como en `desinstalar_demonio.sh`.

6.4 Seguridad

El servicio se instala con `sudo`, pero se ejecuta como `User=hector`. Así se aplica el principio de mínimo privilegio: un error en el script solo podría afectar los archivos de ese usuario, no el sistema completo.

7. Conclusiones

- Con `systemd`, cualquier programa puede convertirse en demonio sin cambiar su código: basta un archivo de unidad.
- `Restart=always` recuperó el servicio automáticamente, con un nuevo PID y el contador de reinicios en 1. Esto evidencia el papel de supervisor de `systemd`.
- Un proceso que espera bloqueado casi no consume CPU. Usar `sleep` o eventos en lugar de espera activa es clave para servicios eficientes.
- `systemctl status` y `journalctl` dan visibilidad completa del ciclo de vida del servicio: estado, PID, recursos, *cgroup* y el historial de arranques y paradas.

8. Referencias

- `man 5 systemd.service`, `man 5 systemd.unit`, `man 1 systemctl`, `man 1 journalctl`, `man 7 daemon`.
- Silberschatz, A., Galvin, P. y Gagne, G. *Operating System Concepts*, 10.^a ed. Wiley.
- Documentación del kernel Linux: *Control Group v2*.